

📖 OOAD Ch14: Design Patterns (設計模式)

主題重點： CH14 的核心是 **Design Patterns**：把物件導向開發中反覆出現的設計問題與解法，以共同語彙整理成可重用的架構經驗。老師特別強調，Pattern 是 **Architecture / Pattern reuse**，不是 code reuse；真正重點不是背模式，而是知道何時該用、何時不該用，並在真實世界的 deadline、bug、dirty software 中維持 **KISS** 原則，把 Pattern 用在刀口上。

📁 Source Materials

- 來源：使用者提供的 OOAD CH14 Design Patterns 課堂整理內容。
- 內容說明：本筆記融合老師上課音檔講解與簡報 (Slides) 重點；依使用者要求，整理時不省略任何主題、例子、警告與模式內容。

📌 0. 本章總覽：軟體設計模式課堂總筆記

本章主題是 **軟體設計模式 (Design Patterns)**。課堂內容包含：

- 設計模式的起源與歷史。
- 老師對設計模式實務使用的警告：不要濫用、不要過度設計。
- 物件導向設計中比封裝、繼承、多型更進階的三大核心原則。
- 三個主要模式深度解析：**Strategy Pattern**、**Singleton Pattern**、**Composite Pattern**。
- GoF 其他常見模式補充：**Proxy Pattern**、**Observer Pattern**、**Visitor Pattern**。
- 結語：以《駭客任務》Morpheus 梗圖「Welcome to the real world」呼應真實世界軟體設計精神。

📖 1. 設計模式的起源與歷史

1.1 什麼是 Design Pattern?

在 **物件導向程式設計 (OOP)** 近 30 年的歷史中，工程師遇到的問題大多不是新的。

Pattern 就是把過去在各種應用領域中解決問題的「寶貴經驗」，用一種共通、大家都能接受的詞彙與方式記錄下來。

換句話說，Design Pattern 不是某個單一程式碼片段，而是把反覆出現的問題與成熟解法抽象化，讓不同開發者可以用共同語言溝通設計。

1.2 歷史淵源：Christopher Alexander 與《A Pattern Language》

最早提出 Pattern 概念的是建築師 **Christopher Alexander**。

他撰寫了《A Pattern Language》一書，強調在建築領域中，解決問題的模式在人類歷史上不斷發生。

軟體工程借鑒了這個概念：

- 建築領域會面對空間、結構、使用者行為的反覆問題。
- 軟體領域同樣會面對物件責任、變動封裝、物件合作、擴充性與維護性的反覆問題。
- 因此，軟體設計也可以把經驗整理成 Pattern language。

1.3 核心本質：Architecture / Pattern reuse, 不是 code reuse

老師強調，**Design Patterns** 的本質是一種：

- 架構上的重用 (**Architecture / Pattern reuse**)
- 而不是 程式碼層級的重用 (**Not code reuse**)

也就是說，Design Pattern 重用的是設計思路、角色配置、責任分配與物件合作方式，而不是直接複製貼上一段程式碼。

1.4 四人幫 (Gang of Four, GoF)

1995 年，**Erich Gamma** 等四人出版了經典著作：

Design Patterns: Elements of Reusable Object-Oriented Software

這四位作者被稱為：

- Gang of Four (GoF)**
- 中文常稱「四人幫」

這本書收集了 **23 種常見的設計模式**，被喻為軟體工程界的「四書五經」。

老師提到：

如果你能看懂這本書，代表你的程式碼已經寫得夠多了。

這句話的意思是，Design Patterns 並不是初學者硬背名詞就能理解的東西。它需要足夠多的實作經驗，才會真正體會為什麼這些模式能解決維護、擴充與變動的問題。

⚠️ 2. 老師的實務箴言與模式的「黑暗面」

老師在課堂中花了大量時間警告學生：

不要濫用設計模式。

這也是業界常見的弊病。

2.1 過度設計 (Over-engineering / Overuse)

很多狂熱的程式設計師，也就是所謂 **Patterners**，為了炫技或自嗨，會硬把設計模式套用到不需要擴充的簡單問題上。

這種做法會造成：

- 程式碼充滿間接呼叫 (indirection)。

- 架構變得深奧難懂。
- 執行效率下降。
- 維護者看不懂原本只是簡單問題的程式。
- 本來可以直接解決的問題，被包成一堆 class / interface / delegation。

老師強調：

過度使用比完全不使用還要糟糕。
The OVERUSE is worst than NO USE it at all.

雖然句子中英文有 typo，但課堂意思非常明確：Pattern overuse 比不用 Pattern 更危險。

2.2 KISS 原則：Keep it simple and stupid

老師提醒，寫程式的第一要務是：

Keep it simple and stupid.

也就是 KISS 原則。

除非你非常確定未來的確有擴充需求，否則請專注於直接解決問題。

Pattern 不是越多越好，而是要在：

- 真的有變動點
- 真的有擴充需求
- 真的需要隔離責任
- 真的能降低未來維護成本

時才引入。

2.3 Welcome to the Real World：歡迎來到現實世界

老師用「Welcome to the Real World」提醒學生：真實世界的軟體開發不是教科書中的完美世界。

在真實世界中，程式設計師有：

- 嚴格的 deadline
- 解不完的 bug
- 不完整的需求
- 需要妥協的設計
- 會變動的客戶要求

老師說，軟體是：

骯髒（dirty）的。

不存在純粹且完美的架構。

一組：

可接受、可用（acceptable, usable）的程式碼

往往比追求：

極致完美（optimal）

更有價值。

2.4 Pattern 的代價：要用在刀口上

任何設計模式的引入都需要付出代價，例如：

- 增加 class 數量。
- 增加 interface / abstract class。
- 增加 indirection。
- 增加理解成本。
- 增加 debugging 路徑。
- 可能增加執行效率成本。

因此，Pattern 需要有針對性地使用在「刀口上」。

優秀的軟體設計師不是看到問題就套 Pattern，而是先問：

- 這個地方真的會變嗎？
- 這個變動值得抽象化嗎？
- 抽象化後是否降低維護成本？
- 現在導入 Pattern 的複雜度是否合理？
- 若不用 Pattern，是否反而更簡單、更安全？



3. 物件導向的三大核心設計原則（OO Principles）

在學習設計模式前，必須先掌握幾個超越基本 Encapsulation、Inheritance、Polymorphism 的進階原則。

3.1 封裝變動的部分（Encapsulate what varies）

原則：

找出應用程式中可能需要變化的地方，把它們獨立出來封裝，不要和那些不需要變化的程式碼混在一起。

目的：

- 把程式修改帶來的副作用降到最低。
- 當變動發生時，只需要改被封裝的部分。
- 穩定的核心邏輯不會因為變動需求而一起被污染。

這也是許多 Design Patterns 的共同核心：把會變的地方抽出去，讓不變的地方保持穩定。

3.2 針對介面撰寫程式，而非針對實作（Program to an interface, not an implementation）

宣告變數時，應該使用 **Supertype**，也就是基礎類別或介面，而不是具體實作類別。

例如：

```
Animal animal = new Dog();
```

JAVA

而不是：

```
Dog d = new Dog();
```

JAVA

這樣核心程式碼才具有 **Polymorphism** 的彈性。

好處：

- 程式依賴抽象，而不是具體類別。
- 未來可以替換不同子類別或實作。
- client code 不需要知道底層物件的具體型別。
- 擴充時可以減少對既有程式碼的修改。

3.3 多用合成，少用繼承（Favor composition over inheritance）

老師強調：使用 **HAS-A**（有一個）的關係將兩個類別結合起來，往往比使用 **IS-A**（是一個）的繼承關係更能獲得維護上的彈性。

也就是說：

- **Inheritance** 是靜態、編譯期決定的關係。
- **Composition** 可以在執行時期替換組合的物件。
- 合成讓行為可以委派給其他物件，降低父類別被迫包含所有行為的問題。

這個原則會在 **Strategy Pattern** 中非常明顯。

🦆 4. 核心設計模式深度解析（一）：Strategy Pattern（策略模式）

4.1 使用情境：SimUDuck 鴨子模擬器

公司開發了一款螢幕保護程式，裡面有多種鴨子，例如：

- **MallardDuck**
- **RedheadDuck**
- 其他不同種類的 duck

這些鴨子原本都繼承自父類別：

```
Duck
```

JAVA

父類別 **Duck** 可能包含共通行為，例如顯示外觀、游泳等。

4.2 錯誤解法 1：直接在父類別加入 fly()

主管要求鴨子要會「飛」。

工程師直覺地直接在父類別 **Duck** 中加入：

```
fly()
```

JAVA

結果引發災難：

- 塑膠鴨（**RubberDuck**）也會飛了。
- 木頭假鴨也會飛了。

但這些鴨子本來不應該會飛。

工程師只好在塑膠鴨類別裡把 **fly()** 覆寫成「什麼都不做」。

問題是：

- 這在維護上是惡夢。
- 每增加一種不能飛的鴨子，就要記得覆寫。
- 父類別放入所有行為，會讓不適用的子類別被迫繼承錯誤行為。
- 這違反了「封裝變動」與「多用合成少用繼承」。

4.3 錯誤解法 2：使用介面 Flyable / Quackable

工程師接著把飛和叫的行為抽出成介面：

```
Flyable
Quackable
```

JAVA

讓需要的鴨子去實作。

這看起來解決了「不該飛的鴨子會飛」的問題，但又產生新問題：

- Java 介面沒有實作程式碼。
- 這導致完全無法達到 **程式碼重用**（**Code reuse**）。
- 如果有 48 種鴨子飛行方式一樣，程式碼就要複製貼上 48 次。
- 這會造成大量 duplicate code。
- 一旦要修改相同行為，就必須改 48 個地方。

因此，單純用 interface 也不是最佳解。

4.4 Strategy Pattern 的正確解法

Strategy Pattern 的做法是：

1. 將會變動的行為抽離出來。
2. 建立獨立的行為類別群組。
3. Duck 不再自己實作所有行為，而是把行為委派給行為物件。

例如建立 `FlyBehavior` 介面：

```
interface FlyBehavior {
    void fly();
}
```

JAVA

底下有不同實作：

```
class FlyWithWings implements FlyBehavior
class FlyNoWay implements FlyBehavior
class FlyRocketPowered implements FlyBehavior
```

JAVA

其中：

- `FlyWithWings`：翅膀飛。
- `FlyNoWay`：不會飛。
- `FlyRocketPowered`：火箭噴射器飛。

4.5 在 Duck 中使用合成（Composition）

在 `Duck` 類別中，不再把 `fly` 行為寫死，而是宣告：

```
FlyBehavior flyBehavior;
```

JAVA

然後在 method 中委派：

```
void performFly() {
    flyBehavior.fly();
}
```

JAVA

這就是 **委派（Delegate）**。

`Duck` 擁有一個 `FlyBehavior`，也就是 HAS-A 關係，而不是用繼承硬塞所有可能行為。

4.6 執行時期動態改變行為

Strategy Pattern 的重要優點是可以在 **Runtime（執行時期）** 動態改變行為。

例如，原本不會飛的鴨子，後來裝上火箭噴射器：

```
duck.setFlyBehavior(new FlyRocketPowered());
```

JAVA

這樣就能讓鴨子改用火箭飛行，而且完全不需要修改 `Duck` 類別本身的程式碼。

4.7 Strategy Pattern 的優點

Strategy Pattern 的優點包括：

- 封裝變動的行為。
- 使用 composition 取代不適合的 inheritance。
- 行為可以重用，不需要 duplicate code。
- 可以 runtime 替換策略。
- `Duck` 類別不需要知道具體飛行演算法。
- 新增行為時，不需要修改原本 `Duck` hierarchy。

4.8 Strategy Pattern 定義

定義：

定義一個演算法家族，將每一個演算法封裝起來，並讓它們可以互換。策略模式讓演算法的變更獨立於使用它的客戶端。

英文可理解為：

Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

5. 核心設計模式深度解析（二）：Singleton Pattern（單例模式）

5.1 動機：系統中只需要唯一一個物件個體

Singleton Pattern 用於系統中某些類別只需存在「唯一一個」物件個體的情況。

常見例子：

- 印表機的 `Spooler`
- 檔案系統 (File system)
- 圖形編輯器裡的單一畫布 (Canvas)
- 圖形編輯器裡的對話框 (Dialog)

這些物件在系統中如果出現多個，可能會造成狀態不一致、資源衝突或管理混亂。

5.2 為什麼不用全域變數 (Global Variables) ？

初學者常喜歡用全域變數，但老師說它是：

邪惡的 (EVIL)

全域變數缺點包括以下幾點。

5.2.1 Non-locality (非區域性)

程式碼到處都能修改全域變數。

這會導致：

- 難以追蹤誰改了它。
- 難以除錯。
- 程式狀態可能在任何地方被破壞。

5.2.2 無存取控制

全域變數缺乏約束，容易引發安全性問題。

任何地方都能直接讀寫，沒有清楚的控制點。

5.2.3 隱性耦合 (Implicit coupling)

不同模組表面上看似沒有關係，但其實都依賴同一個全域變數。

這會讓系統產生隱性耦合。

5.2.4 命名空間污染 (Namespace pollution)

全域變數會污染 namespace。

大型專案中，名字衝突、語意不清與管理困難會越來越嚴重。

5.2.5 Concurrency issues (並行安全問題)

多執行緒下，全域變數極度危險。

如果多條執行緒同時讀寫，就可能發生 race condition 或資料不一致。

5.2.6 Eager initialization 的資源浪費

全域變數通常在程式一啟動就必須建立，也就是：

Eager initialization

如果該物件很耗資源卻沒被用到，就會浪費效能與記憶體。

5.3 Singleton 的標準寫法

Singleton 的標準寫法包含三個重點。

5.3.1 把建構子設為 private

把 constructor 設為 `private`，防止外部使用 `new` 建立物件。

```
private Singleton() {}
```

JAVA

5.3.2 類別內部保存唯一實體

在類別內部宣告一個 `private static` 變數來保存實體。

```
private static Singleton uniqueInstance;
```

JAVA

5.3.3 提供 public static getInstance()

提供一個 `public static` 的 `getInstance()` 方法。

當被呼叫時：

- 如果實體是 `null`，才進行實例化。
- 否則直接回傳現有實體。

```
public static Singleton getInstance() {  
    if (uniqueInstance == null) {  
        uniqueInstance = new Singleton();  
    }  
    return uniqueInstance;  
}
```

JAVA

這稱為：

延遲初始化 (Lazy Initialization)

好處是只有真正需要時才建立物件。

5.4 多執行緒 (Multithreading) 的挑戰

當兩條執行緒同時進入 `getInstance()` 時，可能會不小心建立出兩個物件。

例如：

1. Thread A 檢查 `uniqueInstance == null`，結果是 true。
2. Thread A 尚未建立完成。
3. Thread B 也進入，檢查 `uniqueInstance == null`，仍然是 true。
4. Thread A 和 Thread B 各自建立一個 Singleton。

這就破壞了 Singleton 的唯一性。

5.5 解法一：synchronized getInstance()

第一種解法是在 `getInstance()` 方法加上 `synchronized` 關鍵字：

```
public static synchronized Singleton getInstance() {
    if (uniqueInstance == null) {
        uniqueInstance = new Singleton();
    }
    return uniqueInstance;
}
```

JAVA

優點：

- 簡單。
- 可以避免多執行緒同時建立多個物件。

缺點：

- 每次呼叫都要同步。
- 效能差。
- 實際上只有第一次建立物件時需要同步，後面每次同步都是多餘成本。

5.6 解法二：Double-Check Locking + volatile

第二種解法是：

雙重檢查鎖定 (Double-Check Locking)

並搭配 Java 的 `volatile` 關鍵字。

`volatile` 的作用是：

- 確保變數不會被執行緒做本地快取。
- 強制直接讀寫主記憶體。
- 避免某些重排序或 visibility 問題。

典型寫法：

```
public class Singleton {
    private volatile static Singleton uniqueInstance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (uniqueInstance == null) {
            synchronized (Singleton.class) {
                if (uniqueInstance == null) {
                    uniqueInstance = new Singleton();
                }
            }
        }
        return uniqueInstance;
    }
}
```

JAVA

這樣只有在第一次建立物件時才進行同步化，大幅提升效能。

🌿 6. 核心設計模式深度解析（三）： Composite Pattern（合成模式）

6.1 使用情境：圖形編輯器與 UML Editor

在圖形編輯器，例如 [UML Editor](#) 中，常會用小零件拼出複雜圖形。

小零件可能包括：

- 線條 (`Line`)
- 文字 (`Text`)
- 基本圖形
- 物件框
- 連接線

並且可以將多個物件：

群組 (Group)

起來成為大物件。

大物件又可以再被群組，形成樹狀結構。

這種結構就是 Composite Pattern 很適合處理的情境。

6.2 樹狀結構：Leaf 與 Composite

在 Composite Pattern 中，通常有：

- **Leaf**（樹葉節點）：不能再包含子物件，例如 `Line`、`Text`。
- **Composite**（複合節點）：可以包含子物件，例如 `Group`。
- **Component / Graphic**（共同基礎類別）：定義 Leaf 與 Composite 的共同介面。

6.3 不良設計：Add / Remove 只放在 Group 類別中

直覺上，開發者會把 `Add()` 和 `Remove()` 方法只寫在 `Group` 類別中。

理由似乎很合理：

- 單一物件 `Line` 不能新增子物件。
- 只有 `Group` 才能新增或移除子物件。

但這會導致 bad OO design smells。

6.4 Bad OO Design Smells：instanceof 與型別判斷

當你要對畫面上被選取的物件做 `Ungroup`（解散群組）時，如果 `Add()` / `Remove()` 只在 `Group` 裡，client code 就被迫寫：

```
if (selected instanceof Group) {
    Group g = (Group) selected;
    g.remove(...);
}
```

JAVA

這代表程式碼必須判斷型別並轉型。

問題：

- 違反物件導向的特異化行為原則。
- client code 需要知道物件到底是 Leaf 還是 Group。
- 型別判斷散落在程式中，未來難以維護。
- 新增圖形種類時，client code 可能又要被迫修改。

老師指出，這種 `instanceof` 是不良設計味道。

6.5 Composite Pattern 的正確解法：向上滲透（Percolating up）

正確做法是將所有管理方法與共通操作：

- `Add`
- `Remove`
- `GetChild`
- `Draw`

通通 向上提升（Percolating up）到最上層基礎類別：

```
Graphic
Component
```

JAVA

也就是讓 Leaf 和 Composite 都有相同的操作介面。

6.6 基礎類別提供空實作或預設實作

在基礎類別中，為這些方法提供：

- 空實作（什麼都不做）
- 或預設實作
- 或在某些設計中拋出 exception

例如：

```
class Graphic {
    void add(Graphic g) {
        // default: do nothing
    }

    void remove(Graphic g) {
        // default: do nothing
    }

    Graphic getChild(int i) {
        return null;
    }

    void draw() {
        // default behavior
    }
}
```

JAVA

6.7 Leaf 節點：Line / Text

對於樹葉節點，例如：

- `Line`
- `Text`

它們繼承後：

- 不覆寫 `Add()` / `Remove()`。

- 或僅拋出例外。
- 實作自己的 `Draw()`。

Leaf 不需要知道 group 的管理細節。

6.8 Composite 節點：Group

對於複合節點，例如 `Group`：

- 實作 `Add()`。
- 實作 `Remove()`。
- 實作 `GetChild()`。
- 在 `Draw()` 方法中，透過迴圈呼叫底下所有子節點的 `Draw()`。

概念如下：

```
class Group extends Graphic {
    List<Graphic> children;

    void add(Graphic g) {
        children.add(g);
    }

    void remove(Graphic g) {
        children.remove(g);
    }

    void draw() {
        for (Graphic child : children) {
            child.draw();
        }
    }
}
```

JAVA

6.9 Composite Pattern 的優點

優點：

- client code 不再需要區分單一物件或群組物件。
- 可以直接一視同仁地呼叫共同介面。
- 消滅 `instanceof`。
- 很適合處理樹狀結構。
- group 可以包含 leaf，也可以包含其他 group。
- 可以用遞迴方式處理 draw / move / delete / group / ungroup 等操作。

核心精神：

Treat individual objects and compositions uniformly.

✂ 7. 其他常見模式簡介（The Big Five 補充）

老師在投影片與口述中也提到了 GoF 另外幾個常與上列並稱的模式。

7.1 Proxy Pattern（代理模式）

7.1.1 目的

`Proxy Pattern` 的目的，是延緩一個物件的建立與初始化：

On demand

也就是等到真正需要時才建立或載入真實物件。

這通常是因為該物件的建立可能很耗費：

- 速度
- 記憶體
- I/O 成本
- 網路成本

7.1.2 使用情境：文件中的高解析度影像

例如，一份文件中含有許多高解析度影像。

剛開啟文件時，不需要一次把所有圖片全部載入。

若全部載入會造成：

- 開啟速度慢。
- 記憶體使用量大。
- 使用者可能根本不會捲到那些圖片。

7.1.3 作法

Proxy 物件具備與真實物件相同的介面。

client 端呼叫 Proxy。

Proxy 在真正需要時，才去讀取磁碟並載入真正的 `Image`。

流程：

Client → ProxyImage → RealImage

Proxy 在一開始可以只保留：

- 圖片路徑
- 尺寸資訊
- 佔位資訊

等到需要顯示時，才真正建立或載入 `RealImage`。

7.2 Observer Pattern（觀察者模式）

7.2.1 目的

`Observer Pattern` 的目的，是定義物件間「一對多」的依賴關係。

當一個物件，也就是：

Subject

狀態改變時，所有依賴它的物件，也就是：

Observers

都會自動收到通知並更新。

7.2.2 常見應用：MVC

Observer Pattern 常見應用於：

MVC (Model-View-Controller) 架構

同一份數據，也就是 `Model`，可以同時更新不同畫面，例如：

- 圓餅圖
- 長條圖
- 表格
- 其他 view

當 Model 的資料改變時，多個 View 都會收到通知並更新。

這樣可以讓 Model 不需要知道具體有哪些 View，只要通知 observers 即可。

7.3 Visitor Pattern（存取者模式）

7.3.1 目的

`Visitor Pattern` 適用於你有一個複雜物件結構，例如編譯器的：

抽象語法樹 (AST, Abstract Syntax Tree)

你希望對裡面的節點進行各種不同的新操作，例如：

- 型別檢查 (type checking)
- 產生程式碼 (code generation)
- 排版列印 (pretty printing)

但你不想把這些操作的程式碼全部塞進節點類別中。

7.3.2 作法

Visitor Pattern 的做法是將操作邏輯抽出成獨立的 `Visitor` 類別。

節點類別只需要單純接受：

```
accept(visitor)
```

JAVA

也就是讓 Visitor 來拜訪節點。

這樣可以：

- 避免破壞節點原有資料結構。
- 把不同操作分散到不同 Visitor 類別。
- 當新增操作時，不必把所有邏輯塞回 AST node 裡。

概念：

```
AST Node.accept(Visitor)
Visitor.visit(SpecificNode)
```

7.3.3 使用意義

Visitor Pattern 特別適合：

- 物件結構相對穩定。
- 但需要常常新增對結構的操作。

例如 compiler 中，AST node 種類可能穩定，但對 AST 的操作可能很多：

- semantic analysis
- optimization
- code generation
- pretty print
- visualization

8. 課堂中的模式與 OO 原則對照

模式 / 原則	解決的問題	主要技術	老師提醒
Encapsulate what varies	變動污染穩定程式碼	抽出變動部分	先找出真正會變的地方
Program to an interface	client 依賴具體實作	Supertype / interface	宣告用 <code>Animal</code> ，不要綁死 <code>Dog</code>
Favor composition over inheritance	繼承造成行為僵化	HAS-A / delegate	Duck 的 fly / quack 應交給 behavior object
Strategy Pattern	演算法或行為需要互換	behavior family + composition	可 runtime 換行為
Singleton Pattern	系統只需要唯一物件	private constructor + static instance	多執行緒要小心
Composite Pattern	Leaf / Group 需要一致處理	Component + tree recursion	消滅 <code>instanceof</code>
Proxy Pattern	真實物件建立成本太高	proxy 與 real object 同介面	on demand 載入
Observer Pattern	一對多狀態更新	Subject / Observer	MVC 常用
Visitor Pattern	對複雜結構新增操作	accept / visit	AST 常見

9. 老師語氣中特別重要的考試 / 實務提醒

9.1 Pattern 不是拿來炫技

設計模式不是為了讓程式看起來比較高級。

如果一個簡單問題不需要擴充，硬套 Pattern 會讓程式更難懂。

9.2 Pattern 的使用一定有代價

每一個 Pattern 都會增加某種成本。

常見成本：

- class 變多
- call chain 變長
- indirection 變多
- debugging 更困難
- 初學者更難看懂

因此要問：這個成本是否換來足夠的維護彈性？

9.3 真實世界需要 acceptable / usable，不一定是 optimal

老師說真實世界是 dirty 的。

所以工程師要能在：

- 時間
- 品質
- 複雜度
- 維護性
- deadline
- bug pressure

之間取得平衡。

9.4 KISS 永遠優先

先保持簡單。

只有在未來變動與維護需求足夠明確時，再引入 Pattern。

✓ 本章核心觀念總結

- Design Patterns** 是把過去反覆出現的 OO 設計問題與成熟解法整理成共同語彙。
- Pattern 起源可追溯到 Christopher Alexander 的建築 Pattern language；GoF 在 1995 年整理了 23 個經典 OO design patterns。
- Design Pattern 是 **Architecture / Pattern reuse**，不是單純 code reuse。
- 老師反覆警告：不要濫用設計模式，**OVERUSE** 比 **NO USE** 更糟。
- KISS** 原則是本章最重要的實務提醒：保持簡單，除非真的有擴充需求。
- 三大 OO principles：封裝變動、針對介面寫程式、多用合成少用繼承。
- Strategy Pattern** 用 SimUDuck 說明：不要把會變的 fly / quack 寫死在父類別；應抽成 behavior family 並用 composition / delegation。
- Singleton Pattern** 用於系統只需唯一物件的情況；要避免全域變數的 non-locality、implicit coupling、namespace pollution 與 concurrency issues。
- Singleton 在多執行緒下需要注意 synchronized、double-check locking 與 `volatile`。
- Composite Pattern** 用於圖形編輯器 / UML Editor 的 group / ungroup 樹狀結構；將 `Add`、`Remove`、`GetChild`、`Draw` percolate up 到 base class，可消滅 `instanceof`。
- Proxy Pattern** 延遲高成本物件建立；**Observer Pattern** 支援一對多更新與 MVC；**Visitor Pattern** 支援在 AST 等複雜結構上新增操作。
- 本章精神是「Welcome to the real world」：學 Pattern 是為了解決真實世界長期維護與變動的痛點，不是為了堆砌華麗架構。

原始內容保留對照

本筆記已完整納入使用者提供內容中的所有主要段落與細節：

- 「設計模式的起源與歷史」：Design Pattern 定義、Christopher Alexander、《A Pattern Language》、Architecture / Pattern reuse、GoF 23 patterns、老師對 GoF 書的說法。
- 「老師的實務箴言與模式的黑暗面」：Over-engineering / Overuse、Patterns、indirection、OVERUSE worst than NO USE、KISS、Welcome to the Real World、dirty software、acceptable / usable vs optimal、Pattern 要付出代價並用在刀口上。
- 「OO Principles」：Encapsulate what varies、Program to an interface not implementation、Animal animal = new Dog() vs Dog d = new Dog()、Favor composition over inheritance、HAS-A vs IS-A。

- 「Strategy Pattern」：SimUDuck、MallardDuck、RedheadDuck、Duck、直接加入 fly() 導致 RubberDuck / 木頭假鴨會飛、覆寫為不做事的維護災難、Flyable / Quackable interface 無 code reuse、48 種鴨子 duplicate code、FlyBehavior、FlyWithWings、FlyNoWay、FlyRocketPowered、composition、delegate、runtime 更換行為、Strategy 定義。
- 「Singleton Pattern」：Spooler、檔案系統、Canvas、Dialog、global variables evil、non-locality、無存取控制、implicit coupling、namespace pollution、concurrency issues、eager initialization、private constructor、private static instance、public static getInstance、lazy initialization、multithreading、synchronized、double-check locking、volatile、主記憶體讀寫與效能。
- 「Composite Pattern」：圖形編輯器、UML Editor、Line、Text、Group、群組再群組、tree structure、Add / Remove 只放 Group 的 bad smell、Ungroup、instanceof Group、違反 OO 特異化行為、percolating up、Graphic / Component、Add / Remove / GetChild / Draw、空實作、Leaf、Composite、Group draw 迴圈、消滅 instanceof。
- 「Big Five 補充」：Proxy、Observer、Visitor；高解析度影像 on demand、Proxy 與真實物件同介面、MVC、Model 同時更新 pie chart / bar chart、AST、type checking、code generation、pretty printing、Accept / Visitor。
- 「結語」：Morpheus 梗圖、Welcome to the real world、不要寫疊床架屋的華麗程式、解決長期維護與變動痛點、KISS、評估未來擴充可能性、Pattern 花在刀口上。

Related Notes

-  [School/OOAD_Object-Oriented Analysis and Design](#)
- [School/OOAD/CH13_Methods_and_Sequence_Diagram](#)
- [School/OOAD/TermProject_UML_Editor](#)
- [Strategy Pattern](#)
- [Singleton Pattern](#)
- [Composite Pattern](#)
- [Proxy Pattern](#)
- [Observer Pattern](#)
- [Visitor Pattern](#)